

[דף סיכום בחינה](#)

מזהה סטודנט: N102281665

מזהה קורס: 20905 שם קורס: שפות תכנות

מספר שאלה	ניקוד מירבי	ציון
1	35.00	34.00
2	35.00	35.00
3	30.00	30.00

ציון בחינה סופי : 99.00

הבחינה הבדוקה בעמודים הבאים



לשימוש הבודק

שאלה 1

שאלה: $enum_exp$: 1. פרט את כל המקומות הדרושים.

2. נוסח את כל המקומות.

$enum_exp$: 1. פרט את כל המקומות הדרושים (member).

* נוסח את כל המקומות (identifier-val).

$match_exp$: 1. פרט את כל המקומות הדרושים (exp).

2. פרט את כל המקומות הדרושים (enum).

3. פרט את כל המקומות הדרושים (enum).

4. פרט את כל המקומות הדרושים (enum).

5. פרט את כל המקומות הדרושים (enum).

הקטן language ה: the grammar

(expression
("enum" "{" (separated-list identifier ";") "}")
enum-exp)

(expression
(">" expression "." identifier ">")
enum-elmt-exp)



(expression
("match" ":" identifier "::"
expression ";")
(lambda ("?" identifier "=>" expression ";"))

הקטן data-structures ה: ADT
expval אקספרסיה

(identifier-val (v symbol?))
(enum-val (v enumdt?))

הקטן ADT

(define-datatype enumdt enumdt?
(enum (list-of identifier-val?)))

לא מוגדר

-1


```

(define (expval  $\Rightarrow$  enum val)
  (cases expval val
    (enum-val (v) v)
    (else (eopl:error "Not an enum"))))

(define (expval  $\Rightarrow$  id val)
  (cases expval val
    (identifier-val (v) v)
    (else (eopl:error "Not a variable"))))

(define (identifier-val? val)
  (cases expval val
    (identifier-val (v) #t)
    (else #f)))

```

:010782 0380 : value-of - $\rightarrow$ interpreter Lisp

```

(enum-exp (vars)
  (let* ((lst (map (lambda (v)
                    (identifier-val v)) vars)))
    (if (null? lst)
        (eopl:error "enum-exp" "empty list
of vars")
        (has-dup? lst)
        (eopl:error "enum-exp" "has duplicates"))
    *

```



```
(else (enum-val (enum lst))) )])
```

: Value-of-8 סמן

```
(define (has-dup? lst)
```

```
(cond
```

```
((null? lst) #f)
```

```
((member (car lst) (cdr lst)) #t)
```

```
(else
```

```
(has-dup? (cdr lst))))
```



: Value-of סמן

```
(enum-elt-exp enumexp (enum var)
```

```
(let* ((enum (expval → enum  
          (value-of enumexp env)))
```

```
(idlst (cases enumelt enum
```

```
(enum (v) v)))
```

```
(vars (map (lambda (v)
```

```
(expval → id v)) idlst)))
```

```
(if (not (member var vars))
```

```
(error 'enum-elt-exp "Not  
an enum var")
```

```
(identif-va val var))))
```



(match-exp (enumvar fieldexp fields

```
(match-exp (enumvar fieldexp fields exps)
  (let* ((enumval (apply-env env enumvar))
        (fval enumval (value-of fieldexp env))
        (en (expval⇒enum enumval))
        (vars (cases enumval en
                      (enum (v) v)))
        (pvars (map (lambda (v)
                       (expval⇒id v)) vars)))
    (cond
      ((null? fields)
       (error 'match-exp "empty list given"))
      ((not (member (expval⇒id fval) pvars))
       (error 'match-exp "given var not
                  an enum var"))
      ((or (not (contains? fields pvars))
            (not (contains? pvars fields)))
       (error 'match-exp "no match in
                  variables"))
      (else (let* ((var (expval⇒id fval))
                    (index (index-of var fields 0)) *
                    (exp (list-ref exps index)))
                (value-of exp env))))))
```

value of -1 for non existing

(define (index-of element lst index)

(cond

((null? lst) -1)

((equal? (car lst) element) index)

(else

(index-of element (cdr lst) (+ 1 index))))

דוגמה: רשימת רשימות

(define (contains? lst1 lst2)

(cond

((null? lst1) #t)

((not (member (car lst1) lst2)) #f)

(else

(contains? (cdr lst1) lst2))))



7

[illegible]

("fon/proc" "("identifien"; "

"{" (separated list identifier ";") "}" ;

"[" (opened list expression ",") "]")"

(ambro "3" guard "7")

"{" expression "}")

forproc-exp)

Legend

("skip" ":" expression)

Ship-guard)

(guard

("break" ":" expression)

(break-guard)

10/10

(forproc-exp (loopvar pvars pbody guards body)
(concl

((not (= (length pvars) (length pbody))))

(exp:error 'forproc-exp "Not same len lists")

(= 0 (length pvars))

(num-val 27))

(doe ; ~~and~~ create a procedure p using the
; var and body, and eval the guards exps
; using it. With (map ...)
; get the index of the first true

(let* (new-env (extend-env loopvar
(newref (proc-val (procedure
(car pvars) (car pbody) env))) env))

(vals (map (lambda (e) ^{exps}
(value-of e new-env)) ~~guards~~))
← פסיק חלופי
כאן מסומם 11
exps נגזר
(~~for~~ i (index-of (bool-val #t) vals)))

(concl

((= -1 i) ~~raise~~

(begin

(value-of body new-env)

(value-of (forproc-exp loopvar
(cdr pvars) (cdr pbody) guards body)
env)))

~~raise~~

; ~~end~~ ^{הסיום}

קצת מסורבל
אבל נכון.

~~value~~

(else

(let* ((act ~~map~~ (action-of i guards)))

(if (string=? act "skip")

(value-of ^{for-proc-exp} ~~value-of~~ loop-var (cdr pvars)

(cdr pbody) guards body) env)

; break guard

(num-val 27))))))

(value-of-8 from) guard-0 stop pt 20500

(define (action-of i gs)

(let* ((g (list-ref i gs))

(typ (cases guard g

(skip-guard (v) "skip")

(break-guard (v) "break"))))

typ))

: דרך של סופר 20500 : Value of 8 from

(define (index-of e lst)

(helper e lst 0))

(define (helper e lst i)

(cond

((null? lst) -1)

((equal? (car lst) e) i)

(helper e (cdr lst) (+ 1 i))))

מאגנסים שמים צהרסות מוספר :

(exp (map (lambda (g)
 (cases guard g
 (skip-guard (v) v)
 (break-guard (v) v))))

זה מה שהיה חסר לי כדי להבין בסוף את הפתרון

י בס	לשימוש הבודק
י בס	250180
י בס	12

מחזורי תא

ית
בס

גליון תשובות לשאלות רב-ברריות

הקף במעגל את התשובה שבחרת (לכל שאלה יש רק תשובה אחת נכונה).
 אם תרצה לבטל תשובה שבחרת, סמן עליה X.
 דוגמה לתשובה שבחרת: א ב ג ד ה ו ז ח ט
 דוגמה לתשובה שבטלת: א ב ג ד ה ו ז ח ט

שאלה	תשובה	שאלה	תשובה
1	א ב ג ד ה ו ז ח ט	21	א ב ג ד ה ו ז ח ט
2	א ב ג ד ה ו ז ח ט	22	א ב ג ד ה ו ז ח ט
3	א ב ג ד ה ו ז ח ט	23	א ב ג ד ה ו ז ח ט
4	א ב ג ד ה ו ז ח ט	24	א ב ג ד ה ו ז ח ט
5	א ב ג ד ה ו ז ח ט	25	א ב ג ד ה ו ז ח ט
6	א ב ג ד ה ו ז ח ט	26	א ב ג ד ה ו ז ח ט
7	א ב ג ד ה ו ז ח ט	27	א ב ג ד ה ו ז ח ט
8	א ב ג ד ה ו ז ח ט	28	א ב ג ד ה ו ז ח ט
9	א ב ג ד ה ו ז ח ט	29	א ב ג ד ה ו ז ח ט
10	א ב ג ד ה ו ז ח ט	30	א ב ג ד ה ו ז ח ט
11	א ב ג ד ה ו ז ח ט	31	א ב ג ד ה ו ז ח ט
12	א ב ג ד ה ו ז ח ט	32	א ב ג ד ה ו ז ח ט
13	א ב ג ד ה ו ז ח ט	33	א ב ג ד ה ו ז ח ט
14	א ב ג ד ה ו ז ח ט	34	א ב ג ד ה ו ז ח ט
15	א ב ג ד ה ו ז ח ט	35	א ב ג ד ה ו ז ח ט
16	א ב ג ד ה ו ז ח ט	36	א ב ג ד ה ו ז ח ט
17	א ב ג ד ה ו ז ח ט	37	א ב ג ד ה ו ז ח ט
18	א ב ג ד ה ו ז ח ט	38	א ב ג ד ה ו ז ח ט
19	א ב ג ד ה ו ז ח ט	39	א ב ג ד ה ו ז ח ט
20	א ב ג ד ה ו ז ח ט	40	א ב ג ד ה ו ז ח ט

לשימוש פנימי

מספר התשובות הנכונות: _____ ציון: _____

שם הבודק: _____

250180